

Module Frameworks : test de fin

Enoncé

Objectifs : Fournir un projet Spring Boot qui offre des services de gestion des employés, avec les exigences suivantes

- ✓ Utiliser Spring Boot avec Spring Data JPA et Spring WEB
- ✓ Utiliser la base Oracle
- ✓ Utiliser Maven comme gestionnaire de dépendances

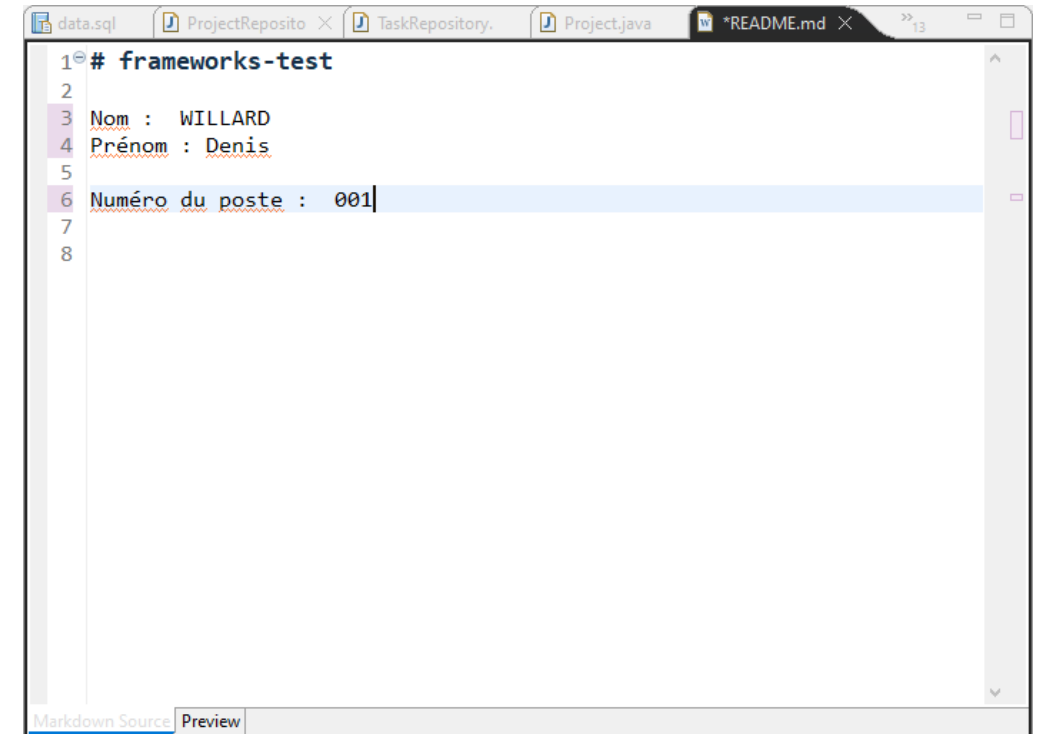
Enoncé

Livrable : Vous rendrez le projet zippé sur l'espace d'échange. Pour éviter toute confusion, merci d'indiquer vos nom/prénom dans le nom du fichier (projet_<nom>_<prenom>.zip)

Pour vous identifier:

Vous indiquerez vos Nom, Prénom, numéro de votre poste de travail

Dans le fichier README.md qui se trouve à la racine (au niveau du pom.xml)



The screenshot shows a code editor window with several tabs: 'data.sql', 'ProjectRepository', 'TaskRepository', 'Project.java', and 'README.md'. The 'README.md' tab is active, displaying the following content:

```
1 # frameworks-test
2
3 Nom : WILLARD
4 Prénom : Denis
5
6 Numéro du poste : 001|
7
8
```

The text is formatted as a Markdown document. The editor has a line number margin on the left and a 'Preview' button at the bottom right.

Enoncé

1 - Initialisation : Fournir un projet Spring Boot qui offre des services de gestion des employés, avec les exigences suivantes

- ✓ Le squelette du projet "test-frameworks" est disponible sur le répertoire d'échange ("frameworks-test.zip")
- ✓ Importer le projet "test-frameworks" dans Eclipse
- ✓ Utiliser la base Oracle
- ✓ Utiliser Maven comme gestionnaire de dépendances

Enoncé

2 - Modélisation : Compléter les classes suivantes pour les transformer en Entités JPA, puis ajoutez les annotations nécessaires pour mettre en place le mapping entre ces classes et les tables Oracle correspondantes

- Employee
- Department
- Address
- Project
- Task

Enoncé

Modélisation (suite) :

Vous mettrez en place des annotations de validation

- @NotNull sur les champs obligatoires
- @Size pour tenir compte de la taille des champs Oracle

Enoncé

Modélisation (suite) :

Vous mettrez en place les annotations Lombok

- @Getter
- @Setter
- @NoArgsConstructor
- @AllArgsConstructor

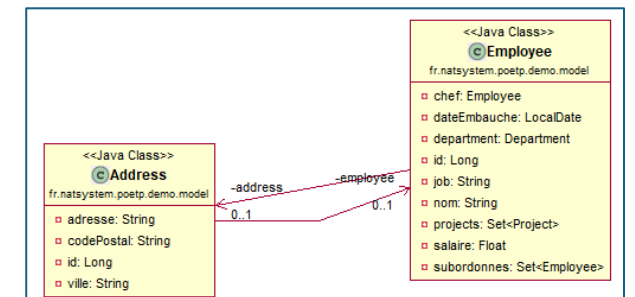
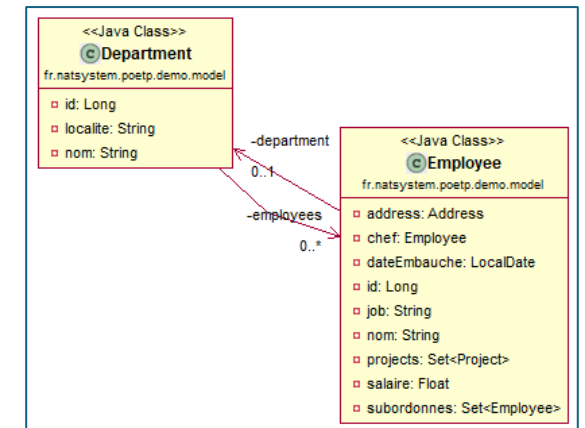
Vous mettrez en place les annotations pour générer automatiquement les ids en avec les séquences Oracle, en utilisant

- @GeneratedValue
- @SequenceGenerator

Enoncé

Modélisation (suite) : Mettez en place les relations suivantes

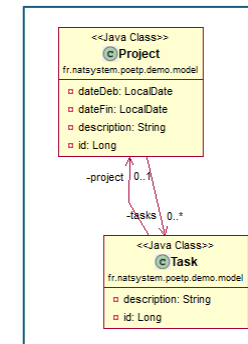
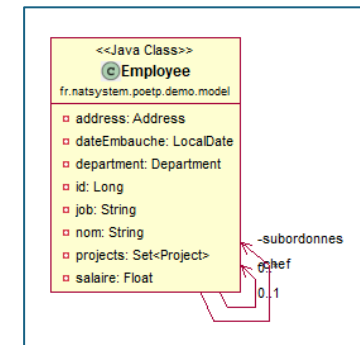
- Dans Employee : le lien @ManyToOne entre Employee et Department
 - **private** Department **department**;
- Dans Department : le lien inverse @OneToMany
 - **private** Set<Employee> **employees**;
- Dans Address : le lien @OneToOne entre Address et Employee
 - **private** Employee **employee**;
- Dans Employee : le lien @OneToOne inverse entre Employee et Address
 - **private** Address **address**;



Enoncé

Modélisation (suite) : Mettez en place les relations suivantes

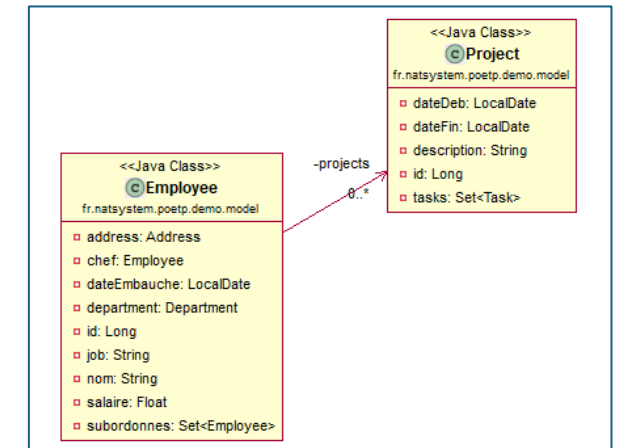
- Dans Employee : le lien @ManyToOne entre Employee et Employee
 - **private** Employee **chef**;
- Dans Employee : le lien inverse @OneToMany
 - **private** Set<Employee> **subordonnes**;
- Dans Task : le lien @ManyToOne entre Task et Project
 - **private** Project **project**;
- Dans Project : le lien inverse @OneToMany entre Project et Task
 - **private** Set<Task> **tasks**;



Enoncé

Modélisation (suite) : Mettez en place les relations suivantes

- Dans Employee : le lien @ManyToMany entre Employee et Project
 - **private** Set<Project> **projects**;
- Note #1 : le lien N-N est par la table Z_PROJECT_PARTICIPATION
- Note #2 : Dans Project on n'implémentera pas le lien @ManyToMany inverse



Enoncé

3 - Création des couches de service et repository :

- Les classes de **repositories Spring Data JPA** (JpaRepository) sont déjà présentes dans le projet pour 5 entités (package "repository")
- Les classe de **services** métiers (Interface + implémentation) psont déjà présentes dansle projet our les 5 entités (package "service")
- Vous créerez les méthodes qui vous semblent nécessaires pour les API demandés à la question **4 – Exposition des API REST**

Enoncé

Création des couches de service et repository :

- 3.1 Dans AdresseServiceImpl vous écrirez l'implémentation de ces service

```
public interface AddressService {  
    ResponseAddressDTO get(final Long id);  
    List<ResponseAddressDTO> findAll(Sort sort);  
    Page<ResponseAddressDTO> findAll(final Pageable pageable);  
    Long create(final RequestAddressDTO dto);  
    void update(final Long id, final RequestAddressDTO dto);  
    void delete(final Long id);  
}
```

Enoncé

Création des couches de service et repository :

- 3.2 Dans DepartmentServiceImpl vous écrirez l'implémentation de ces service

```
public interface DepartmentService {  
    ResponseDepartmentDTO get(final Long id);  
    List<ResponseDepartmentDTO> findAll(Sort sort);  
    Page<ResponseDepartmentDTO> findAll(final Pageable pageable);  
    Long create(final RequestDepartmentDTO dto);  
    void update(final Long id, final RequestDepartmentDTO dto);  
    void delete(final Long id);  
}
```

Enoncé

Création des couches de service et repository :

- 3.3 Dans EmployeeServiceImpl vous écrirez l'implémentation de ces service

```
public interface EmployeeService {  
    ResponseEmployeeDTO get(final Long id);  
    List<ResponseEmployeeDTO> findAll(Sort sort);  
    Page<ResponseEmployeeDTO> findAll(final Pageable pageable);  
    Long create(final RequestEmployeeDTO dto);  
    void update(final Long id, final RequestEmployeeDTO dto);  
    void delete(final Long id);  
}
```

Enoncé

Création des couches de service et repository :

- 3.4 Dans ProjectServiceImpl vous écrirez l'implémentation de ces service

```
public interface ProjectService {  
    ResponseProjectDTO get(final Long id);  
    List<ResponseProjectDTO> findAll(Sort sort);  
    Page<ResponseProjectDTO> findAll(final Pageable pageable);  
    Long create(final RequestProjectDTO dto);  
    void update(final Long id, final RequestProjectDTO dto);  
    void delete(final Long id);  
}
```

Enoncé

Création des couches se service et repository :

- 3.5 Dans TaskServiceImpl vous écrirez l'implémentation de ces service

```
public interface TaskService {  
    ResponseTaskDTO get(final Long id);  
    List<ResponseTaskDTO> findAll(Sort sort);  
    Page<ResponseTaskDTO> findAll(final Pageable pageable);  
    Long create(final RequestTaskDTO dto);  
    void update(final Long id, final RequestTaskDTO dto);  
    void delete(final Long id);  
}
```


Enoncé

4 – Exposition des API Rest :

Pour les contrôleurs REST demandés :

- Vous utiliserez les RequestDTOs et ResponseDTOs qui sont dans le package "dto"
- Les mappers sont fournis (vous les trouverez dans le package "mapper")

Enoncé

Exposition des API Rest (suite) :

Créer des contrôleurs REST pour :

- 4.1 Ajouter, modifier, supprimer et récupérer un Département
- 4.2 Ajouter, modifier, supprimer et récupérer une Adresse
- 4.3 Ajouter, modifier, supprimer et récupérer un Employé
- 4.4 Trouver les Employés dont l'adresse est dans une ville donnée
- 4.5 Trouver les Employés travaillant sur un Projet donné (utiliser une Specification)
- 4.6 Trouver les Employés embauchés en 1982

Enoncé

Exposition des API Rest (suite) :

Créer des contrôleurs REST pour :

- 4.7 Trouver la liste des métiers ("job") des Employés, avec pour chacun de ces métiers le nombre d'employés correspondant (vous pourrez utiliser la classe JobStatsDTO dans le package "projection")
- 4.8 Trouver la liste des employés ayant pour supérieur KING (trié par date d'embauche)
- 4.9 Trouver la liste des employés dont le supérieur a pour supérieur KING
- 4.10 Trouver la liste des supérieurs des employés qui travaillent sur au moins un projet (trié par nom d'employé)
- 4.11 Trouver la liste des employés qui travaillent sur un projet démarré en 2006

Enoncé

Exposition des API Rest (suite) :

Créer des contrôleurs REST pour :

- 4.12 Trouver la liste des employés avec recherche multicritère :
 - ✓ Id
 - ✓ Nom
 - ✓ Job
 - ✓ Date d'embauche
 - ✓ Année d'embauche
 - ✓ Salaire
 - ✓ Nom du supérieur
- Prévoir un "order by"

Enoncé

Exposition des API Rest (suite) :

Créer des contrôleurs REST pour :

- 4.13 Faire une version similaire au 4.12, mais en ajoutant une Pagination

Enoncé

Exposition des API Rest (suite) : Questions Bonus

Créer des contrôleurs REST pour :

- 4.14 Trouver la liste des Employés pour lesquels il existe un responsable qui n'a pas d'adresse
- 4.15 Trouver la moyenne de salaire des employés, par année d'embauche
- 4.16 Trouver les Employés dont l'année d'embauche est supérieure à l'année d'embauche moyenne de tous les Employés

Fin du test